



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO THỰC HÀNH 02

Làm trơn ảnh và phát hiện biên cạnh

Nhóm: Myopia
MSHV 1: 25C11067 - Nguyễn Thái Thông
MSHV 2: 25C11035 - Trần Hạ Khánh Duy

TP. Hồ Chí Minh, 2026

1 Mục tiêu

Bài thực hành tập trung vào hai nhóm thao tác cơ bản trong xử lý ảnh số: làm trơn ảnh và phát hiện biên cạnh. Với mỗi thuật toán, nhóm cài đặt một phiên bản thủ công bằng Python/Numpy, sau đó đối chiếu với hàm tương ứng trong OpenCV.

Mục tiêu chính của bài gồm:

- hiểu cách một kernel được áp dụng lên ảnh thông qua thao tác padding và quét cửa sổ lân cận;
- cài đặt thủ công các bộ lọc làm trơn gồm mean filter, Gaussian filter và median filter;
- cài đặt thủ công các phương pháp phát hiện biên gồm Sobel, Laplacian, Laplace of Gaussian và Canny;
- so sánh kết quả thủ công với OpenCV bằng ảnh trực quan, sai số MSE, thời gian chạy và bộ nhớ đỉnh.

Ảnh đầu vào là ảnh Lenna, được resize về kích thước 150×150 để các thuật toán thủ công chạy nhanh hơn. Các thuật toán làm trơn được áp dụng trên ảnh màu, còn các thuật toán phát hiện biên được áp dụng trên ảnh xám.

2 Phân công đóng góp

Thành viên	Nội dung đóng góp
Nguyễn Thái Thông	Chuẩn bị notebook, cài đặt các hàm tiện ích, phần làm trơn ảnh, đo MSE, lưu kết quả hình ảnh, đối chiếu với OpenCV thời gian chạy và bộ nhớ.
Trần Hạ Khánh Duy	Cài đặt phần phát hiện biên, và hoàn thiện nhận xét báo cáo.

3 Bảng tự đánh giá

Thành viên	Mức đóng góp	Ghi chú
Nguyễn Thái Thông	100%	Hoàn thành phần được phân công, phối hợp kiểm tra kết quả thực nghiệm.
Trần Hạ Khánh Duy	100%	Hoàn thành phần được phân công, phối hợp rà soát notebook và báo cáo.

4 Cơ sở lý thuyết và cách cài đặt

4.1 Tiền xử lý và hàm tiện ích

Notebook sử dụng ảnh Lenna làm ảnh đầu vào. Ảnh được đọc bằng OpenCV ở định dạng BGR, sau đó resize về 150×150 . Với nhóm thuật toán phát hiện biên, ảnh màu được chuyển sang ảnh xám bằng `cv2.cvtColor`.

Để xử lý pixel ở sát biên ảnh, nhóm dùng padding kiểu `reflect`. Cách này phản chiếu giá trị pixel ở vùng biên ra ngoài ảnh, giúp kernel vẫn có đủ vùng lân cận để tính toán. Các hàm tiện ích chính gồm:

- `pad_reflect`: mở rộng ảnh bằng kiểu `reflect`;
- `mse`: tính sai số trung bình bình phương giữa ảnh thủ công và ảnh OpenCV;
- `measure_time_memory`: đo thời gian chạy trung bình và bộ nhớ đỉnh bằng `time.perf_counter` và `tracemalloc`;
- `show_images` và `save_panel_figure`: hiển thị và lưu hình minh họa.

4.2 Mean filter

Mean filter thay giá trị mỗi pixel bằng trung bình cộng của các pixel trong cửa sổ lân cận. Trong notebook, kích thước cửa sổ được chọn là 5×5 . Công thức tổng quát:

$$I'(x, y) = \frac{1}{k^2} \sum_{i=-r}^r \sum_{j=-r}^r I(x+i, y+j)$$

với k là kích thước kernel và $r = \lfloor k/2 \rfloor$. Bản thủ công duyệt từng pixel, lấy vùng lân cận sau padding, tính trung bình theo từng kênh màu rồi đưa kết quả về kiểu `uint8`. Bản OpenCV dùng `cv2.blur` với cùng kích thước kernel và `BORDER_REFLECT_101`.

4.3 Gaussian filter

Gaussian filter cũng là bộ lọc làm trơn, nhưng các pixel gần tâm kernel có trọng số lớn hơn các pixel ở xa. Kernel Gaussian 2D được tạo theo công thức:

$$G(x, y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Sau khi tạo kernel, các trọng số được chuẩn hóa để tổng bằng 1. Trong notebook, nhóm dùng kernel 5×5 với $\sigma = 1.2$ cho phần làm trơn ảnh màu. Bản thủ công tạo kernel bằng Numpy, sau đó áp dụng hàm nhân kernel trên từng vùng ảnh. Bản OpenCV dùng `cv2.GaussianBlur` với cùng kích thước kernel, cùng giá trị sigma và cùng kiểu xử lý biên.

4.4 Median filter

Median filter thay giá trị mỗi pixel bằng trung vị của các pixel trong cửa sổ lân cận. Khác với mean và Gaussian, median không tính trung bình nên ít bị ảnh hưởng bởi các giá trị quá sáng hoặc quá tối. Vì vậy, median thường phù hợp khi ảnh có nhiều dạng điểm.

Trong notebook, bản thủ công dùng cửa sổ 5×5 , lấy trung vị theo từng kênh màu bằng `np.median`. Bản OpenCV dùng `cv2.medianBlur`. Kết quả có sai khác nhỏ do chi tiết xử lý biên và cách tối ưu nội bộ của OpenCV.

4.5 Sobel

Sobel là toán tử đạo hàm bậc một dùng để phát hiện vùng có thay đổi cường độ sáng mạnh. Thuật toán tính gradient theo hai hướng x và y bằng hai kernel:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Trước khi tính Sobel, notebook làm trơn ảnh xám bằng Gaussian 5×5 , $\sigma = 1.0$. Sau đó, độ lớn gradient được tính theo:

$$M = \sqrt{G_x^2 + G_y^2}$$

Bản OpenCV dùng `cv2.Sobel` với kiểu dữ liệu `CV_32F`, sau đó cũng tính độ lớn gradient để đổi chiều với bản thủ công.

4.6 Laplacian

Laplacian là toán tử đạo hàm bậc hai, nhạy với các vùng có thay đổi cường độ sáng nhanh. Vì Laplacian dễ bị nhiễu ảnh hưởng, notebook làm trơn ảnh xám bằng Gaussian trước khi áp dụng kernel Laplacian.

Kernel thủ công được dùng là dạng 4-neighborhood:

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Sau khi tính đáp ứng Laplacian, nhóm lấy trị tuyệt đối và chặn giá trị về đoạn $[0, 255]$ để hiển thị dưới dạng ảnh biên. Bản OpenCV dùng `cv2.Laplacian` với `ksize=1`, tương ứng với kernel 4-neighborhood.

4.7 Laplace of Gaussian

Laplace of Gaussian (LoG) là cách kết hợp làm trơn Gaussian và tính Laplacian. Ý tưởng là giảm nhiễu trước, sau đó mới phát hiện các vùng thay đổi mạnh bằng đạo hàm bậc hai.

Trong notebook này, LoG được triển khai theo đúng chuỗi:

Gaussian smoothing $\rightarrow$ Laplacian

Cụ thể, ảnh xám được làm trơn bằng Gaussian 5×5 , $\sigma = 1.0$, rồi áp dụng kernel Laplacian 4-neighborhood. Bản OpenCV cũng dùng GaussianBlur trước, sau đó dùng cv2.Laplacian với ksize=1. Vì phần Laplacian trong notebook cũng đã có bước Gaussian smoothing trước đó, kết quả Laplacian và LoG trong bảng thực nghiệm giống nhau là điều hợp lý.

4.8 Canny

Canny là phương pháp phát hiện biên gồm nhiều bước hơn các toán tử đơn giản. Trong notebook, bản thủ công mô phỏng các bước chính:

1. làm trơn ảnh xám bằng Gaussian;
2. tính gradient theo hai hướng bằng Sobel;
3. tính độ lớn và hướng gradient;
4. thực hiện non-maximum suppression để làm mảnh biên;
5. dùng double threshold và hysteresis để giữ các biên mạnh, đồng thời nối các biên yếu có liên thông với biên mạnh.

So với phiên bản trước, hysteresis trong notebook này đã được cài bằng DFS, tức là lan truyền từ các điểm biên mạnh sang các điểm biên yếu lân cận. Bản OpenCV dùng cv2.Canny với L2gradient=True để cách tính độ lớn gradient gần hơn với bản thủ công. Tuy nhiên, do Canny phụ thuộc nhiều vào chi tiết non-maximum suppression, ngưỡng và cách nối biên, kết quả thủ công vẫn không thể khớp hoàn toàn với OpenCV.

5 Quy trình thực nghiệm

Quy trình thực nghiệm trong notebook được thực hiện như sau:

1. tạo thư mục data, outputs và doc/figures;
2. tải ảnh Lenna, đọc ảnh bằng OpenCV và resize về 150×150 ;
3. chuyển ảnh màu sang ảnh xám cho các thuật toán phát hiện biên;
4. cài đặt từng thuật toán thủ công bằng Numpy;
5. chạy hàm OpenCV tương ứng với cùng tham số chính;
6. tính ảnh sai khác bằng cv2.absdiff;
7. tính MSE, thời gian chạy và bộ nhớ đỉnh;
8. lưu các panel hình vào thư mục outputs và doc/figures.

Các thuật toán thủ công chủ yếu dùng vòng lặp Python nên chậm hơn OpenCV. Điều này là bình thường vì OpenCV được tối ưu bằng mã nguồn hiệu năng cao. Do `tracemalloc` chủ yếu theo dõi cấp phát bộ nhớ ở phía Python, số liệu bộ nhớ chỉ dùng để tham khảo tương đối, không nên xem là toàn bộ bộ nhớ thật của OpenCV.

6 Kết quả và nhận xét

6.1 Ảnh đầu vào

Ảnh Lenna được dùng ở hai dạng: ảnh màu BGR cho nhóm thuật toán làm trơn và ảnh xám cho nhóm thuật toán phát hiện biên.

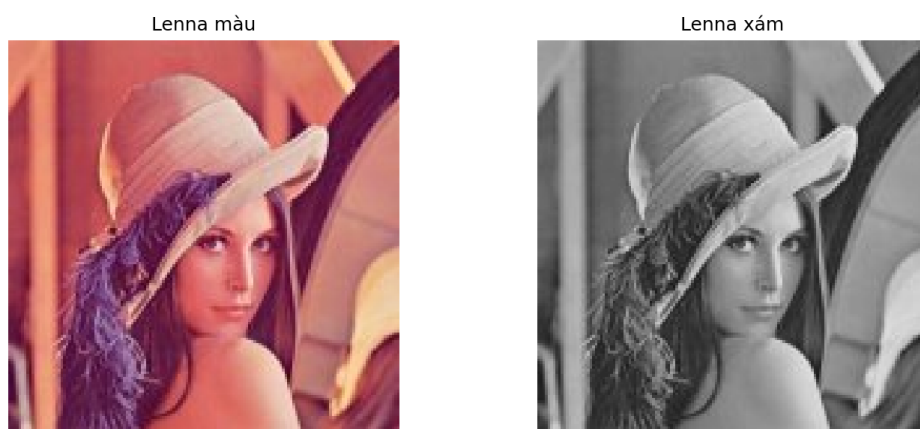


Figure 1: Ảnh Lenna màu và ảnh Lenna xám sau tiền xử lý.

6.2 Mean filter

Mean filter cho MSE bằng 0.0000, tức là kết quả thủ công khớp với OpenCV trong điều kiện thực nghiệm này. Điều này cho thấy cách padding, cách lấy cửa sổ lân cận và cách làm tròn kết quả trong bản thủ công đã tương thích tốt với `cv2.blur`.

Về hiệu năng, bản thủ công mất 0.364940 giây, trong khi OpenCV chỉ mất 0.000063 giây. Sai khác này đến từ việc bản thủ công phải duyệt từng pixel bằng vòng lặp Python, còn OpenCV đã được tối ưu ở mức thấp hơn.

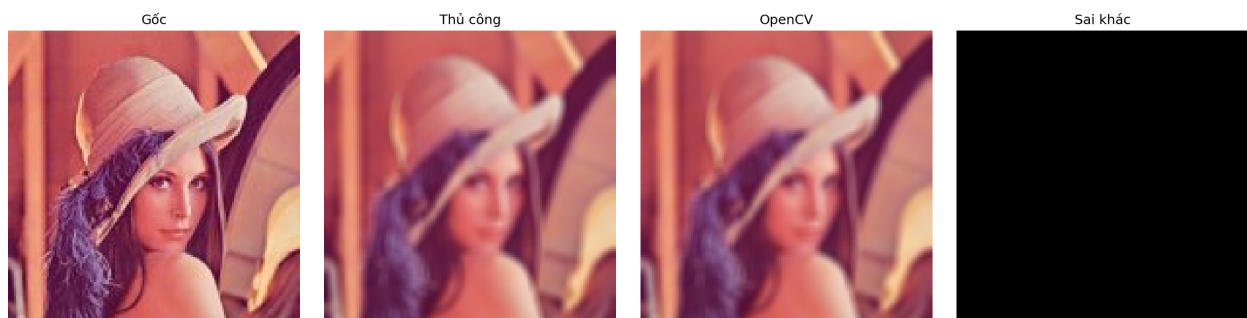


Figure 2: So sánh mean filter thủ công và OpenCV.

6.3 Gaussian filter

Gaussian filter cho MSE bằng 0.0053, rất nhỏ. Kết quả này cho thấy kernel Gaussian thủ công sau khi chuẩn hóa gần như tương đương với `cv2.GaussianBlur`. Sai khác còn lại chủ yếu đến từ làm tròn số thực về `uint8` và một số chi tiết tính toán nội bộ.

Ảnh sau Gaussian nhìn tự nhiên hơn mean filter vì các pixel gần tâm có trọng số lớn hơn. Do đó, ảnh được làm tròn nhưng vẫn giữ cấu trúc chính tốt hơn so với trung bình đều.

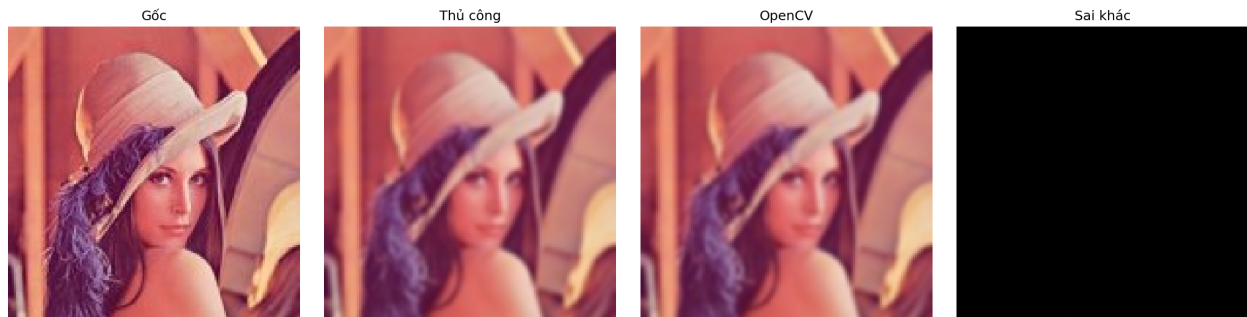


Figure 3: So sánh Gaussian filter thủ công và OpenCV.

6.4 Median filter

Median filter cho MSE bằng 0.4045. Sai số này vẫn nhỏ, nhưng cao hơn mean và Gaussian. Nguyên nhân là median filter phụ thuộc vào cách chọn vùng lân cận, xử lý biên và cách tính trung vị theo từng kênh màu.

Về trực quan, median filter có xu hướng giữ biên tốt hơn mean filter vì nó không lấy trung bình toàn bộ cửa sổ. Tuy nhiên, bản thủ công chạy chậm nhất trong nhóm làm tròn, mất 0.813112 giây, do phải tính trung vị cho từng cửa sổ 5×5 tại từng pixel.

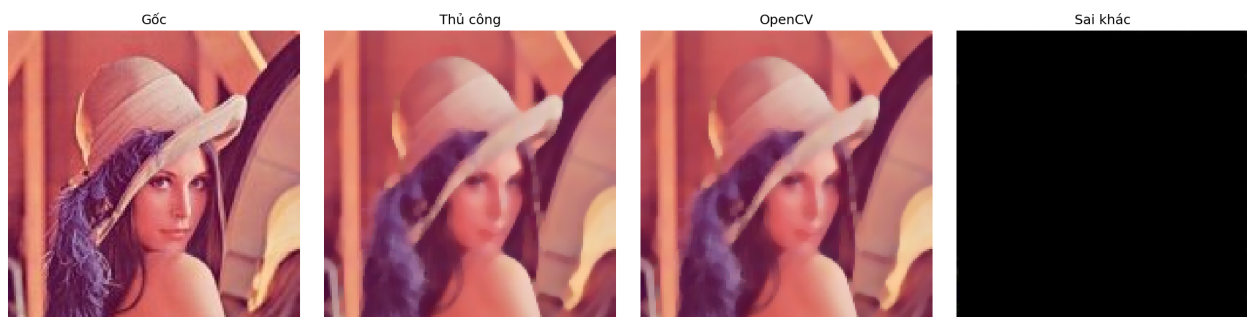


Figure 4: So sánh median filter thủ công và OpenCV.

6.5 Sobel

Sobel cho MSE bằng 0.2713. Đây là mức sai số thấp, cho thấy bản thủ công phát hiện được các biên chính tương đối gần với OpenCV. Các biên quanh khuôn mặt, tóc và vai được thể hiện rõ trong cả hai ảnh.

Bản thủ công làm tròn bằng Gaussian trước, sau đó tính gradient G_x , G_y và độ lớn gradient. Vì quy trình này tương ứng với bản OpenCV nên sai khác không lớn. Phần sai khác chủ yếu đến từ làm

tròn, kiểu dữ liệu và chi tiết xử lý biên.

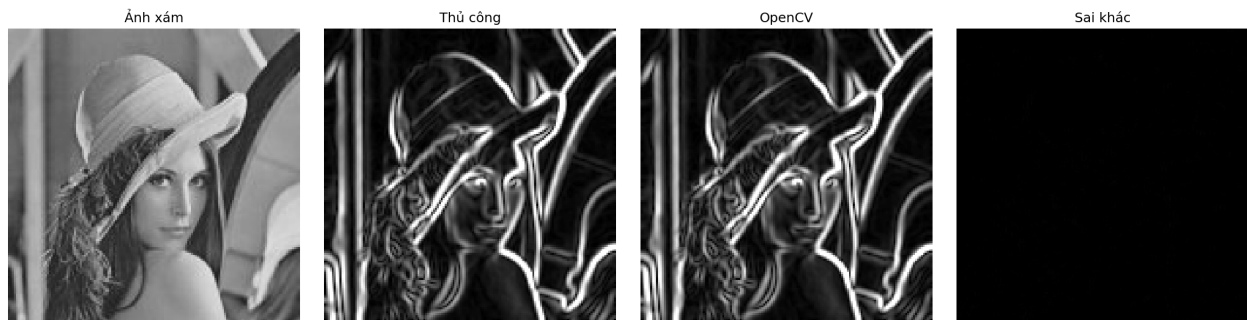


Figure 5: So sánh phát hiện biên Sobel thủ công và OpenCV.

6.6 Laplacian

Laplacian cho MSE bằng 0.3692. Kết quả khá gần OpenCV vì hai bản cùng làm trơn ảnh bằng Gaussian, sau đó dùng Laplacian với cấu hình tương ứng $ksize=1$. Laplacian nhạy với thay đổi cường độ sáng theo nhiều hướng, nên ảnh kết quả có các vùng biên mảnh và nhiều chi tiết hơn so với ảnh làm trơn.

Bản thủ công mất 0.331118 giây, còn OpenCV mất 0.000194 giây. Sự chênh lệch thời gian vẫn chủ yếu đến từ vòng lặp Python trong quá trình nhân kernel.

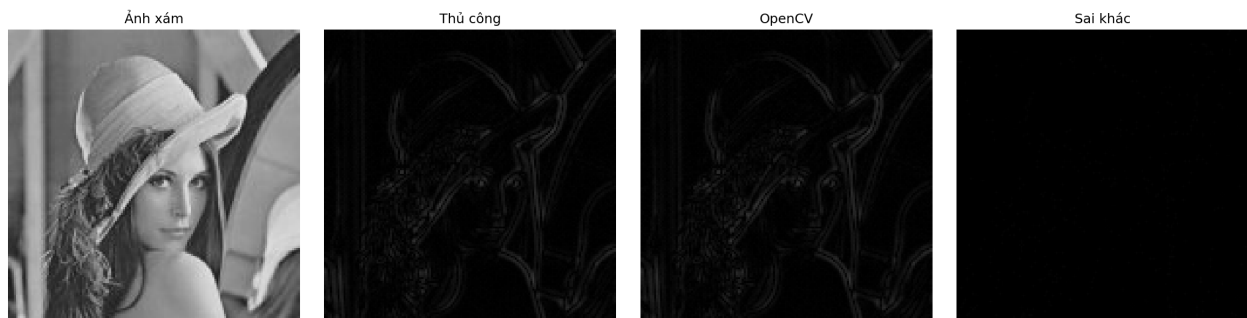


Figure 6: So sánh phát hiện biên Laplacian thủ công và OpenCV.

6.7 Laplace of Gaussian

LoG trong notebook được tính bằng cách làm trơn Gaussian trước rồi áp dụng Laplacian 4-neighborhood. Bản OpenCV cũng dùng Gaussian smoothing trước và `cv2.Laplacian` với $ksize=1$. Vì vậy, MSE của LoG cũng bằng 0.3692, giống phần Laplacian trong notebook.

Về hiệu năng, LoG thủ công mất 0.342633 giây, còn OpenCV mất 0.000196 giây.

Điểm cần lưu ý là trong notebook này, phần Laplacian đã bao gồm bước làm trơn Gaussian trước khi tính đạo hàm bậc hai. Do đó, phần LoG không tạo ra một kết quả khác biệt so với Laplacian, mà chủ yếu làm rõ tên gọi của quy trình Gaussian + Laplacian.

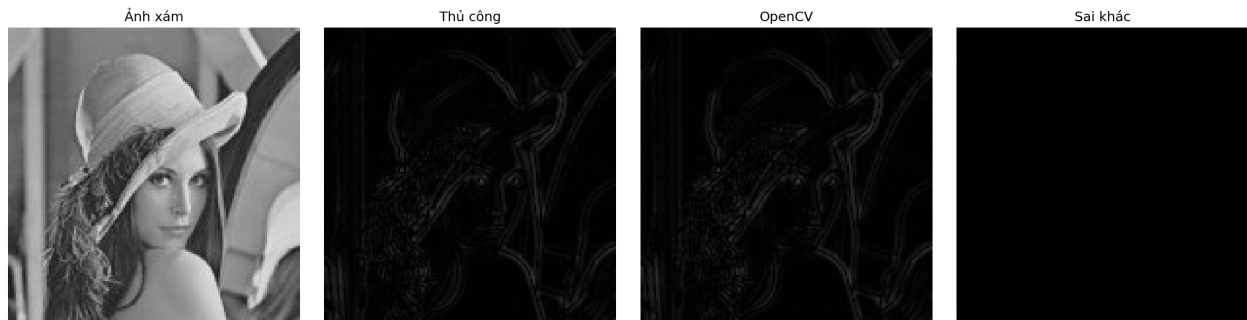


Figure 7: So sánh Laplace of Gaussian thủ công và OpenCV.

6.8 Canny

Nhóm nhận thấy Canny cho MSE bằng 3456.4402, cao hơn nhiều so với các thuật toán trước. Điều này không có nghĩa bản thủ công sai bản chất, vì Canny gồm nhiều bước xử lý chi tiết và rất nhạy với cách cài đặt từng bước.

Bản thủ công đã có đầy đủ các bước chính: Gaussian smoothing, Sobel gradient, non-maximum suppression, double threshold và hysteresis bằng DFS. Tuy nhiên, OpenCV có cách tối ưu và xử lý nội bộ riêng, đặc biệt ở non-maximum suppression, lượng tử hóa hướng gradient và nối biên. Vì vậy, dù dùng `L2gradient=True`, kết quả vẫn có thể lệch đáng kể.

Về trực quan, ảnh thủ công vẫn giữ được các biên chính, nhưng mật độ biên và cách nối biên khác OpenCV. Đây là kết quả hợp lý đối với một bản cài đặt Canny phục vụ mục tiêu học thuật.

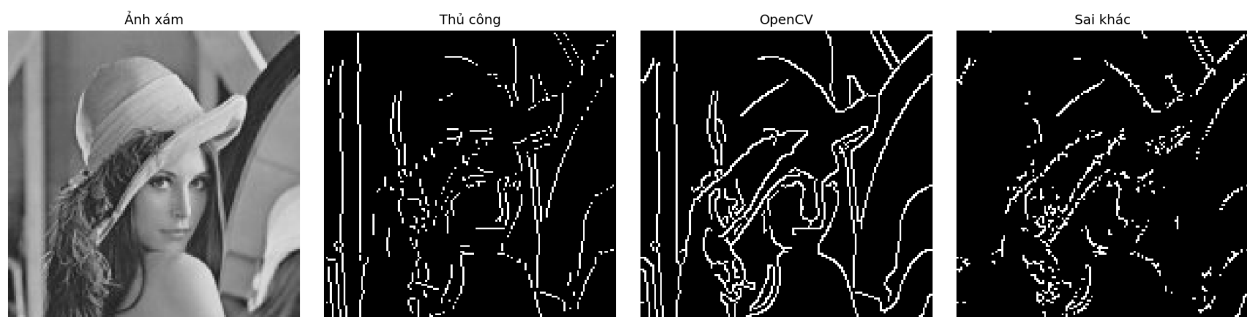


Figure 8: So sánh phát hiện biên Canny thủ công và OpenCV.

6.9 Bảng tổng hợp

Phép xử lý	MSE	Thủ công	OpenCV	Nhận xét ngắn
Mean filter	0.0000	0.364940s 1078.57KB	0.000063s 66.35KB	Khớp hoàn toàn trong thực nghiệm; OpenCV nhanh hơn rõ rệt.
Gaussian filter	0.0053	0.220454s 795.78KB	0.000162s 66.35KB	Sai khác rất nhỏ do làm tròn và chi tiết tính toán.

Median filter	0.4045	0.813112s 213.60KB	0.000180s 66.18KB	Giữ biên tốt, nhưng bản thủ công chậm do tính trung vị từng cửa sổ.
Sobel	0.2713	0.501758s 470.77KB	0.000294s 397.21KB	Phát hiện biên chính rõ, kết quả gần OpenCV.
Laplacian	0.3692	0.331118s 293.39KB	0.000194s 88.54KB	Kết quả gần OpenCV do dùng kernel tương ứng ksize=1.
LoG	0.3692	0.342633s 293.70KB	0.000196s 88.54KB	Tương ứng với chuỗi Gaussian + Laplacian; kết quả giống phần Laplacian đã làm trơn.
Canny	3456.4402	0.543922s 641.65KB	0.000275s 44.40KB	Sai khác cao do Canny phụ thuộc nhiều vào NMS, ngưỡng và hysteresis.

Nhìn chung, các bộ lọc làm trơn và các toán tử phát hiện biên đơn giản cho kết quả gần với OpenCV. Mean filter khớp hoàn toàn, Gaussian và Sobel có sai số rất nhỏ, còn median và Laplacian vẫn nằm trong mức hợp lý. LoG trong notebook này được đối chiếu sát hơn với OpenCV do dùng cùng cấu hình Laplacian ksize=1. Canny là thuật toán phức tạp nhất nên sai khác cao hơn, dù bản thủ công đã mô phỏng đúng các bước chính.

7 Kết luận

Bài thực hành đã hoàn thành việc cài đặt và so sánh các thuật toán làm trơn ảnh và phát hiện biên cạnh. Ở nhóm làm trơn, mean filter, Gaussian filter và median filter đều cho kết quả gần OpenCV. Mean filter làm mờ đều vùng lân cận, Gaussian filter làm trơn tự nhiên hơn nhờ trọng số theo khoảng cách, còn median filter phù hợp hơn khi cần hạn chế ảnh hưởng của nhiễu điểm.

Ở nhóm phát hiện biên, Sobel và Laplacian cho kết quả khá ổn định vì quy trình tính toán tương đối trực tiếp. LoG trong notebook được triển khai dưới dạng Gaussian smoothing kết hợp Laplacian, đồng thời được đối chiếu với OpenCV bằng cùng cấu hình ksize=1, nên kết quả gần với OpenCV. Riêng Canny có MSE cao hơn do thuật toán gồm nhiều bước và phụ thuộc mạnh vào cách hiện thực non-maximum suppression, double threshold và hysteresis.

Qua bài thực hành, nhóm em quan sát rõ hơn cách kernel tác động lên ảnh, vai trò của padding, quá trình tính gradient và cách tạo ảnh biên từ các đáp ứng đạo hàm. Bản cài đặt thủ công có tốc độ chậm hơn OpenCV, nhưng có giá trị trong việc quan sát từng bước xử lý bên trong thuật toán. Trong ứng dụng thực tế, OpenCV vẫn là lựa chọn phù hợp hơn nhờ tốc độ nhanh và kết quả ổn định.